

Table of Contents

1. Executive Summary	1
2. Requirements Map and Recon Method	2
3. Triage Matrix - 20 Repositories	4
4. Adopt Tracks	5
4.1 izaart95-jpg/GLM-Free-API - captcha warmer and token pool (R1, R2, R3)	5
4.2 Goodnessmbakara/agentrouter-opencode-proxy - WAF client-shape pass (R3, R4)	5
4.3 zackiles/cdp-proxy-interceptor - stealth layer and live token capture (R1, R2)	6
4.4 jianlingzhong/opencode-gemini-rotator - pool rotation semantics (R5)	6
4.5 Jobenas/phantom - lightweight login harness (R1)	7
4.6 MrAlony/oc-quota - egress IP rotation (R4, R6)	7
4.7 EfrainGaray/phantomrelay - Phase-3 transport layer (R6)	7
5. Reference Library	8
6. Rejected	9
7. Orchestration Blueprint	9
8. Risk, Compliance and the 7-Day Plan	11

1. Executive Summary

This report investigates 20 unique open-source repositories supplied as candidate tooling for the OWL-DNS-Synergy Phase-2 acceptance path and the Phase-3 transport roadmap. The raw request listed 22 links; two repositories were duplicated (fengkiej/openai-compatible_opencode-fix and huanglong0719/ds2api-browser-proxy), yielding 20 unique candidates. Every repository was reconnoitered live via the authenticated GitHub API on 2026-09-22: metadata (language, stars, last push, activity) plus README deep-reads. Each candidate was scored against six standing requirements, labeled R1 through R6, that cover Google OAuth credential acquisition, token harvest and import, the /v1/messages live smoke test, WAF fingerprint evasion, multi-account pool operations, and the Phase-3 transport upgrades.

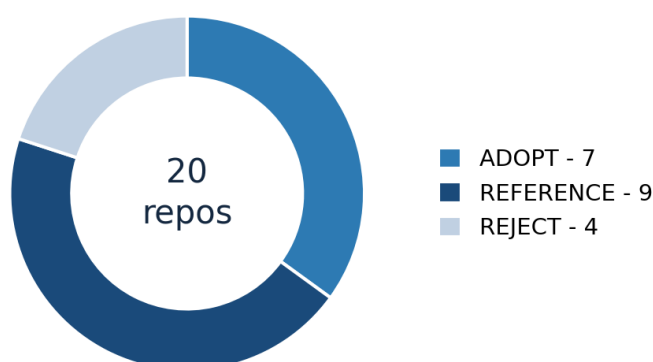


Figure 1: Verdict distribution across the 20 unique repositories (7 adopt, 9 reference, 4 reject).

Finding 1 - the WAF is client-shape-gated, and the pass is documented. Goodnessmbakara/agentrouter-opencode-proxy documents, with a full research trail of linked issues and pull requests, that the Aliyun WAF fronting AgentRouter allowlists requests by client fingerprint at the connection level, not by bearer token. The Python sync anthropic SDK passes the check because it produces a Claude Code-shaped client; generic Node fetch and httpx are rejected with 401 unauthorized-client regardless of key validity. Our own upstream sits behind the same WAF family and shows the same shape of gating (631002 version gate, 400001 middleware rejection). This is the cheapest possible unblock candidate for the Phase-2 smoke test: a one-hour probe with a different client shape, before any browser automation is attempted.

Finding 2 - a working captcha-and-token-pool design exists in our own ecosystem. izaart95-jpg/GLM-Free-API (122 stars, actively pushed) is a pure-HTTP proxy in front of chat.z.ai that ships three mechanisms we failed to build ourselves: background captcha parameter pre-generation (two cached sets, 75 s TTL, auto-pause after 3 min idle, up to 5 retries per captcha), a token collector that harvests device tokens into a tokens.sqlite pool consumed FIFO and deleted after use, and a dual-protocol /v1/messages endpoint with an agent-mode tool-call shim. Our slider solver landed within 0.6 px of target and was still rejected with VerifyCode F001 - a behavioral verdict. Pre-generating challenge parameters before the flow starts is precisely the countermeasure class we were missing.

Finding 3 - the Phase-3 uTLS plan is already built by someone else. EfrainGaray/phantomrelay implements BoringSSL-level TLS JA3/JA4 fingerprinting equal to Chrome 124, exact HTTP/2 SETTINGS values, aged browser profiles with real SQLite history, Bezier and Markov behavioral simulation, and an auto-escalation ladder (http, headless, stealth, human) - with 896 passing tests. This is a near 1:1 realization of the planned uTLS Chrome 120 work with a Rust native addon, matching the intended Rust tls-client direction. It should be adopted behind a feature flag in Phase-3 rather than rebuilt.

The top three immediate actions, in dependency order, are:

- **1. WAF client-shape probe (half a day).** Test the gated upstream routes under three client shapes: the current python-requests UA, the desktop UA, and the sync anthropic SDK shape. If the third passes, promote it to a SmartChannelRouter channel profile and the /v1/messages smoke test may unblock without any browser work.
- **2. Port the captcha warmer and token pool (one to two days).** Re-implement GLM-Free-API background captcha pre-generation and the tokens.sqlite FIFO pool inside autoclaw-autologin, wired to the existing ACLAW_IMPORT_DIR ingest and Fernet-at-rest storage.
- **3. Stand up the CDP stealth-and-capture layer (one day).** Run cdp-proxy-interceptor between the automation client and Chrome with two plugins: one masking Runtime.enable automation surfaces, one capturing tokens live from the logged-in session as an online upgrade of harvest_token.py.

Seven repositories are adopted (three immediately, one in Phase-3, three as pattern ports), nine are retained as reference implementations from which specific mechanisms are borrowed, and four are rejected as unrelated or out of scope. One caution applies across the adopt set: most of these repositories have low star counts and single maintainers, so the integration rule is vendor-and-pin - code reviewed, revisions pinned, executed in an isolated environment - rather than adding them as live dependencies.

2. Requirements Map and Recon Method

The six requirements below were fixed in earlier phases of the program. R1 through R3 form the Phase-2 acceptance critical path: obtain a working AutoClaw credential, import it, and prove the /v1/messages compatibility endpoint against the real upstream. R4 is the standing adversarial constraint that cuts across every stage, while R5 and R6 cover operational maturity and the next transport generation. Any candidate repository had to map to at least one of these requirements to advance past triage.

Req	Focus	Current status (pre-recon)
R1	Google OAuth login automation for 4 accounts with stealth browsing and CAPTCHA handling	Blocked upstream: version gate 631002, middleware rejection 400001, 16+ app-login body variants rejected; slider solver rejected behaviorally (F001)
R2	Token harvest (desktop localStorage or autoclaw2api export), then ACLAW_IMPORT_DIR or POST /api/tokens/import	Harvest kit shipped: harvest_token.py leveldb scanner, PowerShell exporter, import template, guide; import API verified with dedupe and Fernet-at-rest
R3	/v1/messages real-upstream live smoke acceptance	smoke_test_messages_live.py ready with --token-file; awaiting the first valid token
R4	WAF fingerprint evasion (UA override via environment variables; deeper client shaping)	UA override operational; WAF UA check on some routes relaxed since 09-21; TLS-level shaping not yet built
R5	Multi-account rotation, quota and negative-cache operations across 4 accounts	Failure negative cache (60 s TTL) shipped in cache.py; graded cooldowns and invalidation not yet implemented
R6	Phase-3: uTLS Chrome 120 (Rust tls-client), WebSocket fallback, egress rotation, dashboard extension	Design only; no external implementation evaluated before this recon

Table 1: Requirement register R1-R6 and the state of each track entering this recon.

The recon method was deliberately cheap and evidence-first. Metadata and README heads for all 20 repositories were pulled through the authenticated GitHub API in a single persisted script (scripts/repo_recon_20.py), so every verdict in this report is traceable to a captured JSON record. Candidates whose READMEs exposed concrete mechanisms - code paths, configuration contracts, documented upstream probes - received deeper reads; candidates whose descriptions alone disqualified them were not read further. Scoring assigns each repository a 0-5 impact score against the requirement register, and a verdict of ADOPT (integrate or port now, or at the Phase-3 boundary), REFERENCE (borrow specific mechanisms, do not integrate the codebase), or REJECT (no relevant overlap). The user's parallel stealth-browser shortlist - browser-use, agent-browser, ulixee/secret-agent (already cloned locally), thermoptic, and tholian stealth - is treated as the ambient tooling layer; the repositories below are evaluated for how well they complement it.

One prior conclusion frames everything that follows: programmatic email-plus-password login against the upstream was proven impossible live - the version gate rejects the flow, app-login rejects every plausible body shape, and the chat.z.ai SSO realm yields tokens that are invalid for the AutoClaw API (PROFILE 410000, CHAT 401 even when logged in). The agreed path is therefore credential harvesting from a logged-in session. External tooling is valuable exactly insofar as it (a) makes the interactive login survive WAF behavioral checks, (b) captures the token from the logged-in session reliably, and (c) keeps the resulting token pool healthy under rotation.

3. Triage Matrix - 20 Repositories

The matrix below lists every candidate with its ecosystem signals (language, stars, last push), the requirements it maps to, its impact score, and the verdict. The field is notably fresh: the median last-push date falls in 2026, and six repositories were pushed within the two weeks before the recon, which raises confidence that the ported techniques will still match current upstream behavior. Grouping is by verdict: seven adopts, nine references, four rejects.

Repository	Lang	Stars	Pushed	Req	Score	Verdict
izaart95-jpg/GLM-Free-API	JS	122	2026-09-20	R1 R2 R3	5	ADOPT
Goodnessmbakara/agentrouter-opencode-proxy	Py	14	2026-09-20	R3 R4	5	ADOPT
zackiles/cdp-proxy-interceptor	TS	21	2025-02-17	R1 R2	5	ADOPT
jianlingzhong/opencode-gemini-rotator	TS	3	2026-09-15	R5	4	ADOPT
Jobenas/phantom	Py	0	2026-04-25	R1	4	ADOPT
MrAlony/oc-quota	Rust	4	2026-06-05	R4 R6	4	ADOPT
EfrainGaray/phantomrelay	TS	1	2026-08-15	R6	4	ADOPT (P3)
Prodigalgal/any2api	Py/Java	1	2026-09-20	R1 R5	3	REFERENCE
angyedz/QwenFreeApi	JS	37	2026-08-09	R5 R4	3	REFERENCE
huanglong0719/ds2api-browser-proxy	Go	24	2026-05-16	R1 R3	3	REFERENCE
tpaine1737/scraping-skill	Py	0	2026-03-20	R1	3	REFERENCE
javapuppteernodejs/bypass-awsaf-crawl4ai	Py	1	2025-10-22	R4	2	REFERENCE
getphantomsignal/phantomsignal	Py	0	2026-07-26	R4 R6	2	REFERENCE
ZeroHomer/dsh-opencode-zen-bypass	JS	0	2026-08-16	R4	2	REFERENCE
aleks-spv/opencode_retrypush	TS	3	2026-08-22	R5	1	REFERENCE
fengkiej/openai-compatible_opencode-fix	JS	0	2026-03-06	R3	1	REFERENCE
jkeczan/aws-api-gateway-elastic-search-proxy	JS	6	2018-05-17	-	0	REJECT
assaf/zombie	JS	5624	2022-12-09	-	0	REJECT
mirankhan008/sqli-labs	-	1	2022-09-18	-	0	REJECT
alislive/XSSSLayer	Py	1	2026-07-08	-	0	REJECT

Table 2: Full triage matrix. Scores are impact against R1-R6 (0 = none, 5 = direct unblock). ADOPT (P3) = adopt at the Phase-3 boundary.

4. Adopt Tracks

The seven adopted repositories each carry a concrete, named integration point into the existing codebase - not general admiration. Three of them attack the Phase-2 blocker directly (GLM-Free-API, agentrouter-opencode-proxy, cdp-proxy-interceptor); three port operational patterns into the SmartChannelRouter token pool (gemini-rotator, oc-quota, plus QwenFreeApi semantics from the reference set); and one is the Phase-3 transport layer (phantomrelay). Each subsection below states what the project is, why it earned its verdict, and exactly where it lands in our tree.

4.1 izaart95-jpg/GLM-Free-API - captcha warmer and token pool (R1, R2, R3)

The most mature repository in the set (122 stars, pushed the day before recon) is an OpenAI- and Anthropic-compatible proxy in front of chat.z.ai: dual /v1/chat/completions and /v1/messages on one server, SSE with 5 s keep-alive pings, vision uploads through the file endpoint, and an agent-mode shim that translates OpenAI tools into the user-prompt convention the web endpoint accepts, then rewrites the model's TOOL_CALL blocks back into native tool_calls and tool_use with an anti-loop section. It is pure HTTP at runtime - no browser - and stays under challenge pressure by pre-generating captcha parameters in the background and by burning sessions after a single use.

- Port the background captcha pre-generation loop into autoclaw-autologin as a warmer: two cached parameter sets, 75 s TTL, auto-pause after 3 min idle, five retries per captcha. This attacks the F001 behavioral rejection by presenting challenges solved from pre-baked parameters instead of mid-flow attempts.
- Adopt the tokens.sqlite FIFO pool as the store behind ACLAW_IMPORT_DIR: harvest consumes tokens first-in-first-out, deletes after use, and dedupes - complementing the existing Fernet-at-rest encryption.
- Mirror its /v1/messages contract (keep-alive pings, image blocks, tool_use shape, reasoning_effort forwarding) as additional assertions in smoke_test_messages_live.py so Phase-2 acceptance matches the strongest community implementation of the same compatibility surface.

4.2 Goodnessmbakara/agentrouter-opencode-proxy - WAF client-shape pass (R3, R4)

A local proxy (default port 7187) that lets Node.js AI SDK clients use AgentRouter by receiving their requests and re-issuing them through the Python sync anthropic SDK, whose TLS and client signature the Aliyun WAF allowlists. The README carries a documented research trail: the allowlist is confirmed in agentrouter-org/docs issue 21, the identical failure in opencode issue 5060, and the working fix in a Reckora pull request - including live evidence that the OpenAI SDK shape receives 401 unauthorized-client while the Anthropic messages shape passes to 200/503. The same token works or fails depending only on client shape.

- Build scripts/probe_waf_client_shape_matrix.py: replay the gated routes (631002 version gate, 400001 middleware rejection, /v1/messages route) under three shapes - python-requests UA, desktop UA, sync anthropic SDK shape - and record which passes.
- If the sync-SDK shape passes, encode it as a new channel profile in SmartChannelRouter v3: a client-shape dimension alongside the existing UA-override environment lever.
- Reuse the local re-issue proxy as a debug sidecar for acceptance runs, so tooling that cannot speak the allowed shape transparently can still reach the upstream.

4.3 zackiles/cdp-proxy-interceptor - stealth layer and live token capture (R1, R2)

A transparent man-in-the-middle proxy for the Chrome DevTools Protocol with a plugin API to intercept, modify, inject, and filter any CDP message or event between a browser and its automation client. Its headline use case is overwriting Runtime.enable - the call Playwright issues on every page and that modern anti-bot stacks detect - without patching Playwright itself, positioning it as a robust alternative to rebrowser-patches. This is the mechanism layer beneath the user's stealth-browser shortlist: whatever drives the browser, the proxy scrubs the automation surfaces in the middle.

- Insert it between the automation client and Chrome during Google OAuth: one plugin masks automation surfaces (Runtime.enable and context leaks) - the fingerprint class our slider run flagged after landings within 0.6 px still returned VerifyCode F001.
- Write a capture plugin subscribing to Storage and Network domains that emits autoclaw_token.json live from the logged-in session - the online upgrade of the leveldb scanner in the shipped harvest kit.
- Vendor and pin the code (21 stars, single maintainer) and pin Deno 2.1.4 as its test-suite compatibility requires; run it in an isolated profile directory.

4.4 jianlingzhong/opencode-gemini-rotator - pool rotation semantics (R5)

An npm OpenCode plugin that rotates a pool of API keys by wrapping globalThis.fetch transparently: exhausted keys are parked with a cooldown derived from the Retry-After header or the error text, keys returning API_KEY_INVALID are removed permanently, and healthy keys are always preferred, so sessions keep moving without manual intervention. The semantics are exactly what the four-account AutoClaw pool needs, and they generalize the 60-second failure negative cache already shipped in cache.py into graded, per-token state.

- Port the cooldown parser (header and message-derived) into the token pool of SmartChannelRouter: per-token cool-down fields, permanent invalidation on invalid-token responses, healthy-first selection.
- Bind each token to the fingerprint that harvested it, preserving the per-chat fingerprint isolation already scoped for Phase-2 - never rotate a token onto a different identity.

4.5 Jobenas/phantom - lightweight login harness (R1)

A pip-installable stealth browser CLI (phantom-browse) built on Playwright with anti-detection patches: coordinate-based human-like clicks, 80 ms-per-character typing, chained `--fill/--click/--type/--wait` actions, multi-step `--actions plan.json` flows, persistent `--session` reuse, and structured JSON output designed for agent consumption. It is the lightest harness in the shortlist for driving the Google OAuth sequence per account, and its JSON evidence output drops straight into the orchestrator log.

- Encode each account's login as a `plan.json` (email, password, challenge handling) starting with the spare accounts `mymarky9` and `emarkytanky`; keep the primary `mymarky0` account last.
- Persist sessions so re-runs skip login; capture console errors and screenshots as acceptance evidence.
- Vendor and pin (0 stars, single maintainer); retain browser-use and secret-agent as heavier alternatives and thermoptic / tholian stealth as research tracks.

4.6 MrAlony/oc-quota - egress IP rotation (R4, R6)

A Rust-plus-scripts stack that registers N Cloudflare WARP identities via `wgcf`, exposes each as a SOCKS5 proxy through `wireproxy`, and inserts a 429 interceptor between client and upstream that switches pools internally so the calling application never observes a rate limit. A per-provider binding rule keeps non-problematic providers direct. This adds the egress-identity dimension to the Phase-3 transport design and is operationally free - WARP identities cost nothing.

- Adopt the interceptor contract in front of the upstream channels: on rate-limit or WAF block, switch WARP identity and retry internally, returning a clean response to the caller.
- Keep the per-provider binding rule as a router state: some channels stay direct, mirroring the existing HTTP-Preferred / DNS-Fallback three-state design.

4.7 EfrainGaray/phantomrelay - Phase-3 transport layer (R6)

A production-grade stealth browser relay with 896 passing tests: BoringSSL-level TLS JA3/JA4 equal to Chrome 124, exact HTTP/2 SETTINGS values, persistent profiles with aged real-Chrome SQLite history, Bezier mouse and Markov typing simulation, an auto-escalation ladder from plain http through headless and stealth to fully human modes, Docker remote Chrome, and a native MCP server. It is effectively the uTLS Chrome 120 plan of Phase-3 already implemented, including the Rust native addon for JA4 that matches the intended Rust `tls-client` direction.

- Phase-3: integrate the JA4/H2 layer behind a feature flag as the primary egress for WAF-fronted routes; evaluate its Rust addon against the planned Rust `tls-client` rather than building both.
- Map its escalation ladder onto the router's degradation path when a channel is WAF-blocked - http, headless, stealth, human - replacing ad-hoc same-vector retries.

- Use the MCP server as the control surface from the React dashboard for manual escalations.

5. Reference Library

Nine repositories are retained as references: we borrow named mechanisms or confirm design decisions, but their codebases are not integrated. Several are nonetheless architecturally important - any2api in particular is the most complete account-automation control plane in the set, and QwenFreeApi codifies account-pool hygiene rules that our negative cache should grow into. The table records, for each item, what it is and the specific mechanism we take from it.

Repository	What it is	What we borrow
Prodigalgal/any2api	Unified OpenAI-compatible gateway and account automation control plane over 10 web-account providers; Java Spring monolith, Next.js admin, FastAPI automation on Camoufox	RuntimeChannel page-bridge concept; lifecycle vocabulary (register / reauthenticate / keepalive / daily_checkin); AES-GCM credential store; Redis lease and lock model for the 4-account pool
angyedz/QwenFreeApi	Local OpenAI-compatible proxy for Qwen Chat with web UI, SSE streaming and agent tools	Playwright login capture; auto-generated SSXMOD fingerprint cookies; account autorouting with exponential backoff; WAF errors park accounts longer - matches negative-cache tiering
huanglong0719/ds2api-browser-proxy	DeepSeek web through a real Chrome CDP browser proxy; dual-mode model suffix -browser (safe) vs -direct (fast)	Safe/direct dual-mode routing pattern; SSE thinking_content separation mirroring our reasoning_content work; extractUserText message cleaning; React controlled-input pitfalls
tpaine1737/scraping-skill	Agent skill for Claude Code, Cursor, Codex, Gemini CLI and OpenCode; 3-phase scraping workflow with 8 agents	4-level anti-bot escalation ladder as the design rubric; hidden-API recon before browser automation - cheapest path first
javapuppteernodejs/bypass-aws-waf-crawl4ai	Crawl4AI + CapSolver integration guide for AWS WAF (AntiAwsWafTaskProxyLess)	Solve, inject cookie, reload pattern; persistent-context solver extension as the paid fallback; AWS WAF-specific product, technique transfers to Aliyun
getphantomsignal/phantomsignal	OPSEC-native OSINT framework with stealth egress layer (proxy pool, adaptive pacing, JA3/JA4)	Adaptive pacing concept; per-run egress posture reporting; the framework itself is out of scope
ZeroHomer/dsh-opencode-zero-bypass	DSH plugin rewriting the User-Agent via wrapped globalThis.fetch to pass an opencode-CLI client allowlist	Confirms per-host transparent UA rewriting in-process; our environment-variable UA override already covers the need; zero-process alternative to a local proxy
aleks-spv/opencode_retrypush	OpenCode plugin adding a /retry-now command that re-sends the last message of every rate-limited session	Bulk-retry UX concept for the dashboard; marginal for the engine itself
fengkiej/openai-compatible-opencode-fix	npm re-export of the latest @ai-sdk/openai-compatible fixing PDF/audio/text file-part support (opencode issue 16338)	Install only if OpenCode becomes the acceptance client for /v1/messages; unblocks attachment modality tests

Table 3: Reference repositories and the mechanisms borrowed from each.

6. Rejected

Four repositories failed triage cleanly and are documented here so the decision is auditable. Two are unrelated infrastructure or stale technology; two are offensive-security tooling that is out of scope for an access engine and deliberately kept out of the organization's dependency graph for compliance hygiene. None of the four re-enters scope under the current requirement register.

Repository	Stars	Reason for rejection
jkeczan/aws-api-gateway-elastic-search-proxy	6	A 2018 Lambda function proxying search workloads through Elasticsearch; no overlap with any requirement; unmaintained for eight years.
assaf/zombie	5624	Zombie.js is a JSDOM-based headless browser: despite the star count it cannot execute modern OAuth JavaScript stacks, CAPTCHA challenges, or TLS-sensitive flows; unmaintained since 2022-12.
mirankhan008/sqli-labs	1	SQL injection training platform; offensive-education content unrelated to the credential pipeline; excluded on scope.
alisalive/XSSSlayer	1	Offensive real-browser XSS scanner with AI payload generation; a testing weapon, not access infrastructure; excluded on scope and compliance hygiene.

Table 4: Rejected repositories and rationale.

7. Orchestration Blueprint

The adopted tooling orchestrates into five pipeline stages that map onto the existing five-layer engine. Stage A produces the login; Stage B turns the logged-in session into an imported token; Stage C proves the /v1/messages path; Stage D keeps the four-account pool healthy in daily operation; Stage E is the Phase-3 transport generation. The stages are sequential on the critical path (A to C) but operational in parallel afterwards (D, E).

PIPELINE — STAGES A → E MAPPED TO LAYERS L5 → L4 → L3 → L2/L1

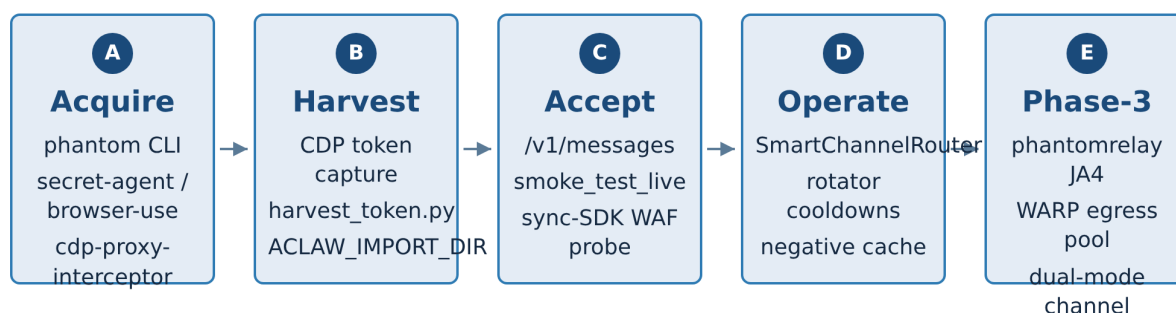


Figure 2: Orchestration pipeline - stages A through E mapped to engine layers L5 down to L1.

Stage A - Acquire (L5 AutoClaw). The phantom CLI drives each Google account's OAuth flow from a plan.json with humanized input and session persistence, beginning with the spare accounts. cdp-proxy-interceptor sits between the automation client and Chrome, masking Runtime.enable and related automation surfaces. CAPTCHA pressure is handled by the ported GLM-Free-API pre-generation warmer; if a challenge still fails, the scraping-skill escalation ladder selects the next vector, and the CapSolver solve-inject-reload pattern is the paid last resort.

Stage B - Harvest and Import (L5 to ingest API). Two capture paths feed the same contract: the CDP capture plugin emits autoclaw_token.json live from the logged-in session, and harvest_token.py remains the offline leveldb scanner for the desktop application. Both emit the token_import_template.json shape; the file is dropped into ACLAW_IMPORT_DIR or posted to /api/tokens/import, landing in the new tokens.sqlite FIFO pool with Fernet-at-rest encryption and dedupe.

Stage C - Accept (L3 core). Run smoke_test_messages_live.py --token-file against the real upstream. In parallel, the WAF client-shape matrix probe determines whether the sync anthropic SDK shape passes the connection-level allowlist; a pass promotes it to a router channel profile and may unblock acceptance before any browser work completes. The UA environment override remains the fast lever, now corroborated by the zen-bypass precedent of in-process per-host UA rewriting.

Stage D - Operate (L4 SmartChannelRouter). The pool gains graded state: Retry-After-derived cooldowns and permanent invalidation from the rotator, WAF-block long-parking from the QwenFreeApi model, and healthy-first selection. Rate-limit failures keep the existing 60-second negative cache; WAF failures park the account-token pair far longer. Pool health is exported to /metrics and visible in the dashboard.

Stage E - Phase-3 Transport (L2/L1). The phantomrelay JA4/H2 layer ships behind a feature flag as the primary egress for WAF-fronted routes, with its escalation ladder wired as the router's degradation path. The oc-quota WARP pool adds egress-identity rotation, and the ds2api dual-mode pattern offers a safe -browser channel suffix for high-risk conversations. The WebSocket fallback remains an internal build item from the original roadmap.

Stage	Engine layer	Our components	External repos
A	L5	autoclaw-autologin, stealth browser stack	phantom; cdp-proxy-interceptor; GLM-Free-API (warmer); scraping-skill (ladder); bypass-awsaf (paid fallback)
B	L5 + ingest	harvest_token.py, ACLAW_IMPORT_DIR, /api/tokens/import, Fernet-at-rest	cdp-proxy-interceptor (capture plugin); GLM-Free-API (token-collector schema)
C	L3	smoke_test_messages_live.py, /v1/messages, UA env override	agentrouter-opencode-proxy (client-shape pass); zen-bypass (UA precedent)
D	L4	SmartChannelRouter v3, cache.py negative cache, /metrics	opencode-gemini-rotator (cooldowns); QwenFreeApi (WAF parking)
E	L2/L1	transport layer, dashboard	phantomrelay (JA4/H2, escalation); oc-quota (WARP egress); ds2api-browser-proxy (dual mode)

Table 5: Stage-to-component mapping with the external repositories engaged at each stage.

8. Risk, Compliance and the 7-Day Plan

The risks concentrate in two places: the accounts themselves and the supply chain. Automated Google login violates Google's terms of service, so the operating posture is deliberately conservative - spare accounts first, low-and-slow pacing, humanized input, and no automation against the primary account until the spare-account path has succeeded end to end. Supply-chain exposure is the mirror risk: most adopted repositories have zero to twenty-one stars and single maintainers, which the vendor-and-pin rule is designed to contain. One housekeeping item surfaced during recon: the GitHub PAT currently lives inside a git remote URL in a local repository config and should move to an environment variable or credential helper, then be rotated after Phase-2 acceptance.

Risk	Severity	Mitigation
Google ToS exposure from automated login	High	Spare accounts first (mymarky9, emarkytanky); low-and-slow pacing; humanized input via phantom; primary account last
Account-ban blast radius across the 4 accounts	High	Per-account fingerprint binding; no credential cross-use; rotation cooldowns; WAF failures park account-token pairs long
WAF escalation from repeated slider failures	Medium	Cap attempts per session; pre-generated captcha parameters; escalate ladder instead of retrying the same vector; log every attempt
Secrets leakage	High	Move PAT from git remote URL into env or credential helper and rotate post-acceptance; keep Fernet token encryption; never commit harvested tokens
Supply-chain risk from 0-21 star repositories	Medium	Vendor and pin revisions; code review before execution; pin Deno 2.1.4 for cdp-proxy; run in isolated venv or container
Compliance drift from offensive tooling	Low	Keep sqli-labs and XSSSlayer outside the org; rejection rationale documented in Chapter 6

Table 6: Risk register with severities and mitigations.

The seven-day plan sequences the work so that the cheapest unblock is attempted first and the expensive browser work only begins once its stealth scaffolding exists. Days 1 through 3 build and probe; day 4 hardens the pool; day 5 is the acceptance run; days 6 and 7 convert the result into user-facing surface and start Phase-3. Each day ends with an artifact recorded in the shared worklog so the next session resumes from evidence rather than memory.

Day	Action	Output / exit criterion
1	Vendor phantom and cdp-proxy-interceptor; author the CDP token-capture plugin; dry-run against the AutoClaw desktop profile	Capture plugin v0 producing a well-formed autoclaw_token.json from a logged-in session
2	Port the GLM-Free-API captcha pre-generation warmer and the tokens.sqlite FIFO pool into autoclaw-autologin	Warmer module + pool store wired to ACLAW_IMPORT_DIR ingest
3	Run the WAF client-shape matrix probe (python-requests UA, desktop UA, sync anthropic SDK shape) on the gated routes	Probe report; go/no-go on promoting the sync-SDK shape to a router channel profile
4	Implement rotator semantics in SmartChannelRouter: Retry-After cooldowns, permanent invalidation, healthy-first selection	Pool v2 with tests green (99 existing + new pool tests)
5	End-to-end spare-account run: login, harvest, import, then smoke_test_messages_live.py --token-file	Phase-2 acceptance evidence: complete success path on /v1/messages
6	Dashboard: claude-* aliases in the model selector; pool-health panel showing cooldowns, WAF parks and quota	Dashboard update merged
7	Phase-3 spikes: phantomrelay JA4/H2 prototype behind a feature flag; WARP egress proof of concept	Spike notes and go/no-go for the Phase-3 transport build

Table 7: Seven-day execution plan with exit criteria.

The net effect of this recon is to convert the Phase-2 blocker from a single uncertain path into three parallel, independently testable paths: a client-shape pass that costs an hour to probe, a captcha warmer plus token pool ported from a battle-tested sibling implementation, and a stealth CDP layer that upgrades the harvest kit from offline scanning to live capture. The standing user workflow is unchanged throughout - harvest the credential, drop it into ACLAW_IMPORT_DIR or POST it to /api/tokens/import, and rerun smoke_test_messages_live.py with --token-file - and every stage above is designed to make that final command succeed on the first fully-evidenced run.